# Hugerth Study: Analysis

```
# =================
# Imports and Setup
# =================

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

import numpy as np
from scipy.stats import gmean

from skbio.stats.distance import DistanceMatrix, permanova
from scipy.spatial.distance import pdist, squareform
from skbio.stats.distance import permdisp

aa_columns = ["Ala","Arg","Asn","Asp","Cys","Gln","Glu","Gly","His","Ile",
    "Leu","Lys","Met","Phe","Pro","Ser","Thr","Trp","Tyr","Val"]
pct_columns = [aa + "_pct" for aa in aa_columns]

samples = pd.read_parquet("hugerth_sample_profiles.parquet")
```

```
# ======================
# Merge metadata and CSTs
# ======================

meta = pd.read_csv("meta_data.csv", encoding="utf-8-sig")
meta_sub = meta[["Run", "Subject", "Visit_No."]].rename(columns={"Run": "sample_id"})
samples = samples.merge(meta_sub, on="sample_id", how="left")

cst = pd.read_pickle("ravel_csts.df").reset_index().rename(columns={"Sample": "sample_id"})
samples = samples.merge(cst[["sample_id", "CST"]], on="sample_id", how="left")

samples = samples.dropna(subset=["CST"]).copy()
samples["CST"] = samples["CST"].astype(int)

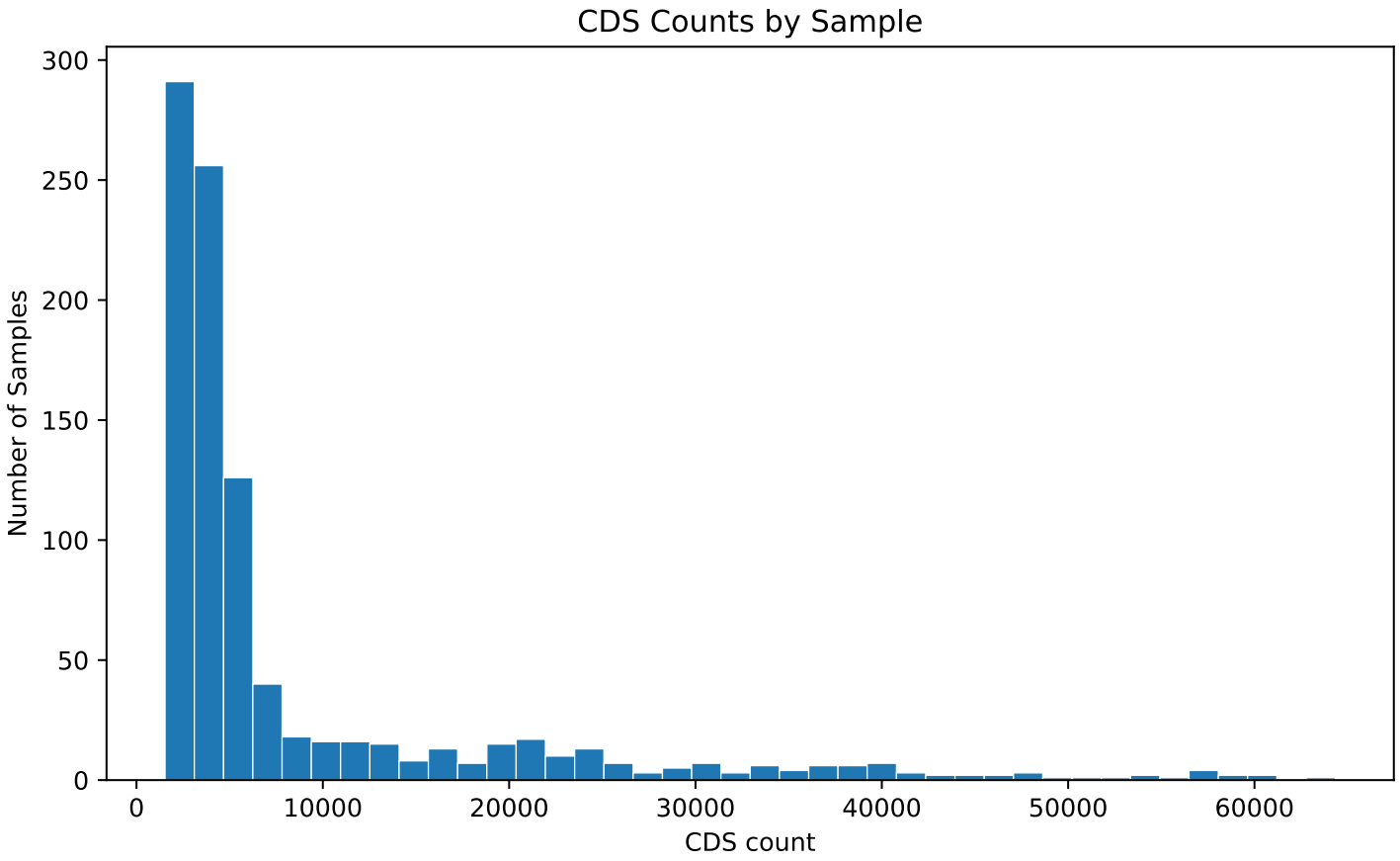
print(samples.shape)
print(samples["CST"].value_counts().sort_index())
```

```
(942, 47)
CST
1    522
3    122
4    293
5      5
Name: count, dtype: int64
```

```
X = samples[pct_columns].values
P = np.where(X == 0, X[X > 0].min()/2, X)
X_clr = np.log(P/gmean(P, axis=1, keepdims=True))
```

```
# ===================
# CDS Count Histogram
# ===================

fig, ax = plt.subplots(figsize=(8, 5))
ax.hist(samples["n_cds"], bins=40, edgecolor="white", linewidth=0.5)
ax.set_title("CDS Counts by Sample")
ax.set_xlabel("CDS count")
ax.set_ylabel("Number of Samples")
plt.tight_layout()
plt.show()
```



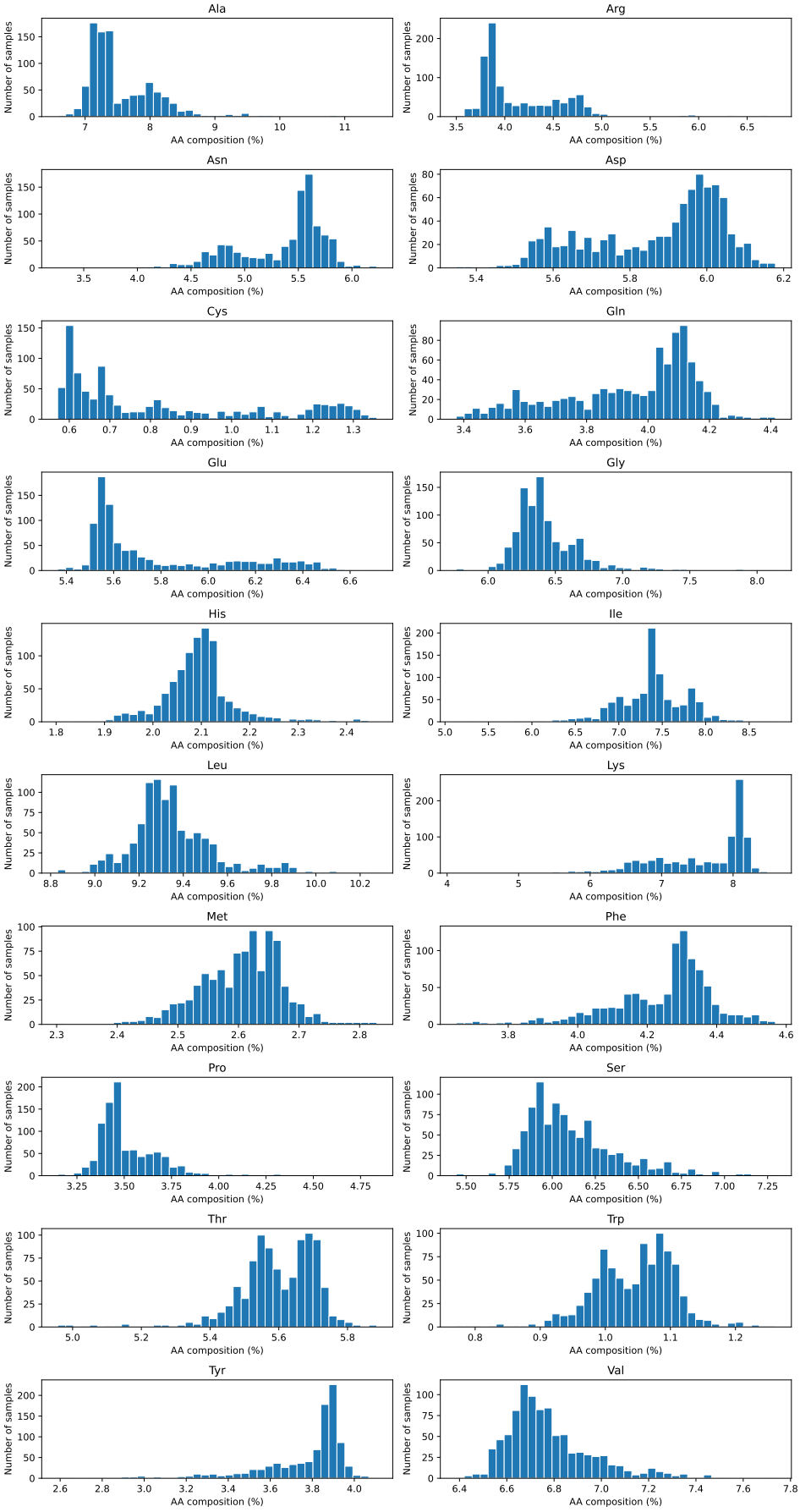

```
# ============================
# Amino Acid Count Histograms
# ============================

fig, axes = plt.subplots(10, 2, figsize=(12, 24))
axes = axes.flatten()

for ax, aa, pct_col in zip(axes, aa_columns, pct_columns):
    ax.hist(samples[pct_col], bins=40, edgecolor="white", linewidth=0.3)
    ax.set_title(aa)
    ax.set_xlabel("AA composition (%)")
    ax.set_ylabel("Number of samples")
```

```
fig.suptitle("AA Profiles by Sample", fontsize=16, y=1.0)
plt.tight_layout(rect=[0, 0.03, 1, 0.98])
plt.show()
```

AA Profiles by Sample

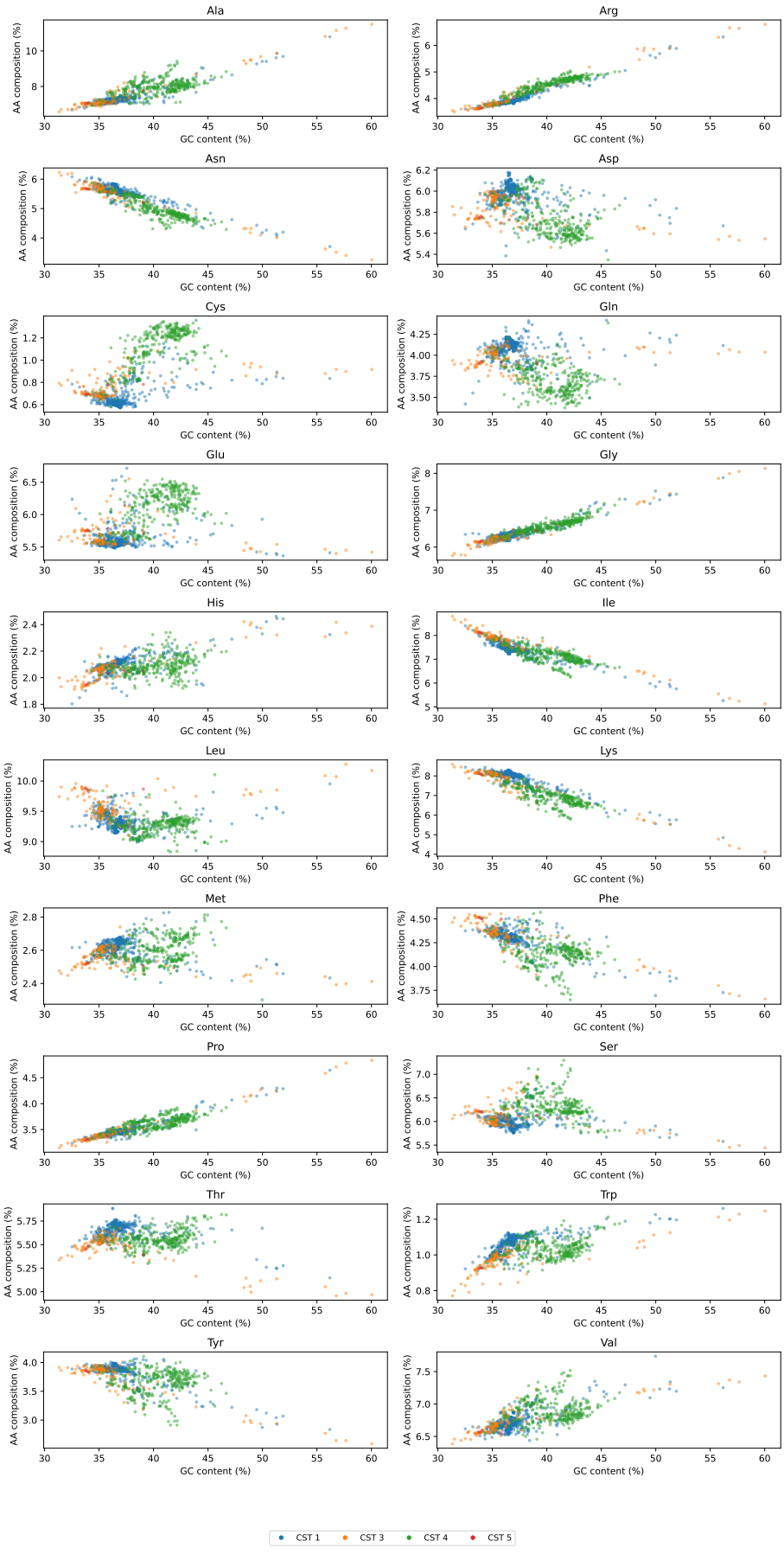


```
# ====================
# GC vs. AA composition
# ====================

cst_levels = sorted(samples["CST"].unique())
cmap_cst = plt.get_cmap("tab10")
cst_colors = {c: cmap_cst(i) for i, c in enumerate(cst_levels)}

fig, axes = plt.subplots(10, 2, figsize=(12, 24))
axes = axes.flatten()
for ax, aa, pct_col in zip(axes, aa_columns, pct_columns):
    for c in cst_levels:
        m = samples["CST"] == c
        ax.scatter(samples.loc[m, "GC_content"], samples.loc[m, pct_col],
                   color=cst_colors[c], alpha=0.5, s=12, linewidths=0)
    ax.set_title(aa)
    ax.set_xlabel("GC content (%)")
    ax.set_ylabel("AA composition (%)")

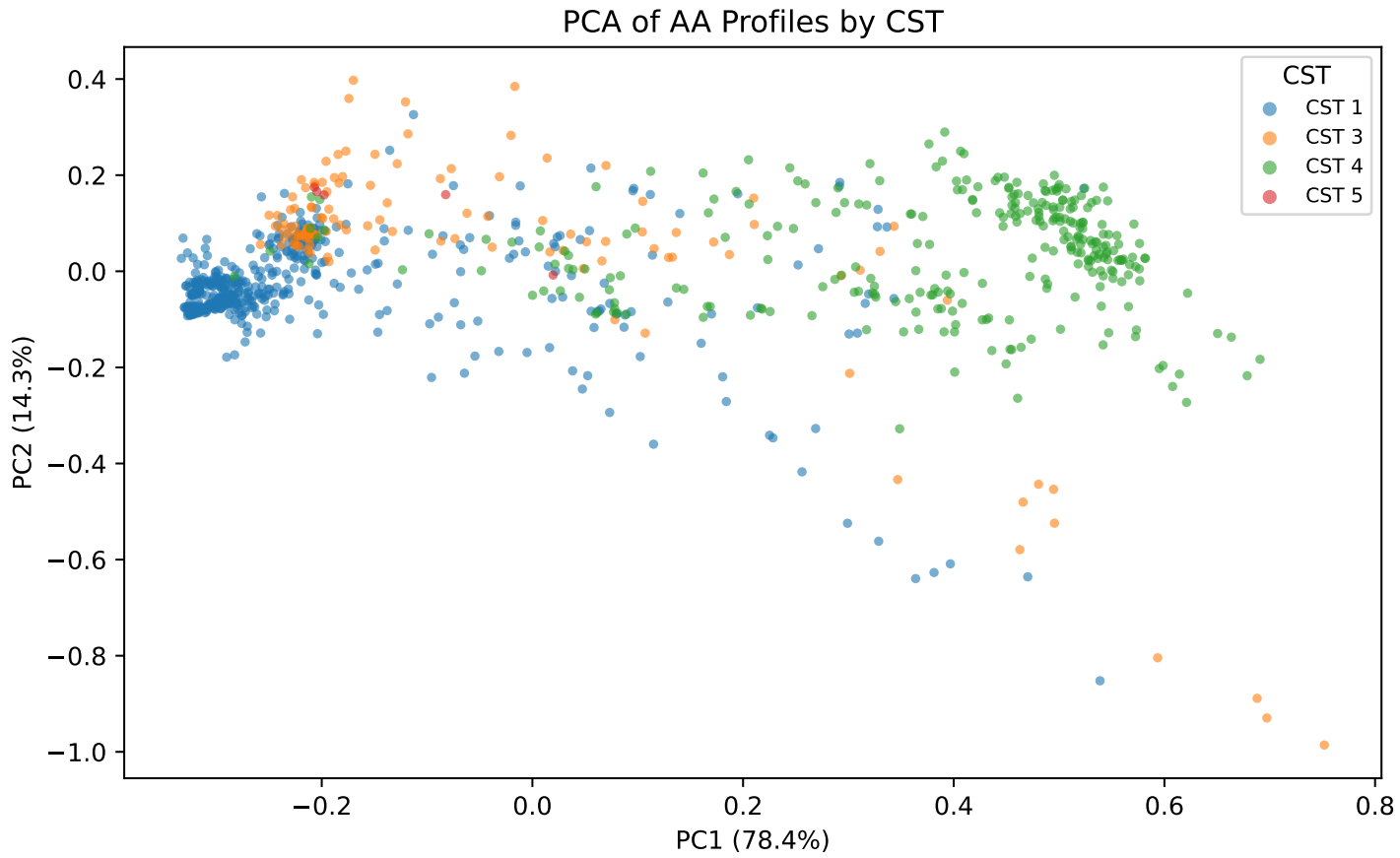
handles = [plt.Line2D([0],[0], marker="o", color="w",
           markerfacecolor=cst_colors[c], markersize=6, label=f"CST {c}")
           for c in cst_levels]
fig.legend(handles=handles, loc="lower center", ncol=len(cst_levels), fontsize=9,
           bbox_to_anchor=(0.5, -0.01))
fig.suptitle("GC Content vs. Amino Acid Profile by CST", fontsize=16, y=1.0)
plt.tight_layout(rect=[0, 0.03, 1, 0.98])
plt.show()
```


GC Content vs. Amino Acid Profile by CST
Ala
Arg
Asn
Asp
Cys
Gln
Glu
Gly
His
Ile
Leu
Lys
Met
Phe
Pro
Ser
Thr
Trp
Tyr
Val
AA composition (%)
GC content (%)
CST 1
CST 3
CST 4
CST 5

```
# ======================
# PCA color coded by CST
# ======================

pca = PCA()
X_pca = pca.fit_transform(X_clr)

fig, ax = plt.subplots(figsize=(8, 5))
for c in cst_levels:
    m = samples["CST"] == c
    ax.scatter(X_pca[m, 0], X_pca[m, 1], color=cst_colors[c],
               alpha=0.6, s=14, linewidths=0, label=f"CST {c}")
ax.set_title("PCA of AA Profiles by CST")
ax.set_xlabel(f"PC1 ({pca.explained_variance_ratio_[0]:.1%})")
ax.set_ylabel(f"PC2 ({pca.explained_variance_ratio_[1]:.1%})")
ax.legend(title="CST", markerscale=1.5, fontsize=8)
plt.tight_layout()
plt.show()
```




```
# ==============
# PCA Scree Plot
# ==============

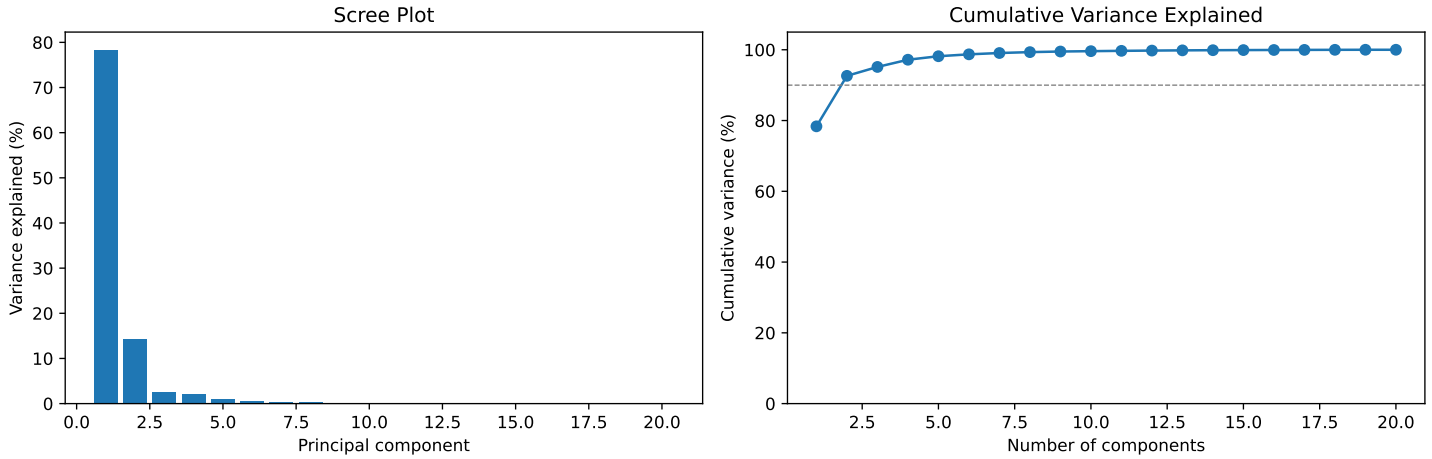
evr = pca.explained_variance_ratio_
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].bar(range(1, len(evr)+1), 100*evr)
axes[0].set_title("Scree Plot")
axes[0].set_xlabel("Principal component")
```

```
axes[0].set_ylabel("Variance explained (%)")

axes[1].plot(range(1, len(evr)+1), 100*evr.cumsum(), marker="o")
axes[1].axhline(90, linestyle="--", linewidth=0.8, color="gray")
axes[1].set_title("Cumulative Variance Explained")
axes[1].set_xlabel("Number of components")
axes[1].set_ylabel("Cumulative variance (%)")
axes[1].set_ylim(0, 105)

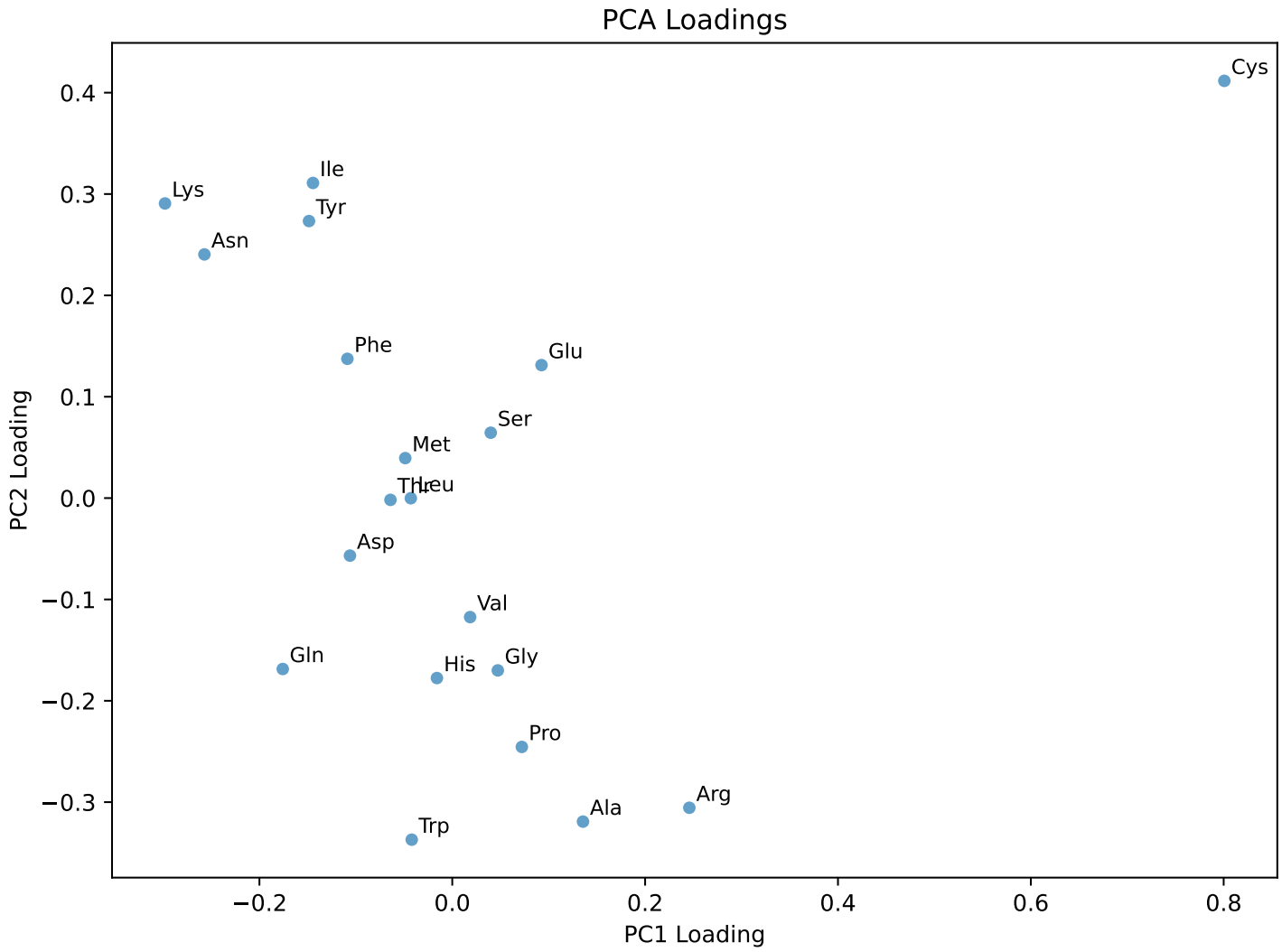
plt.tight_layout()
plt.show()
```




```
# ================
# PCA Loading Plot
# ================

load = pca.components_[:2].T

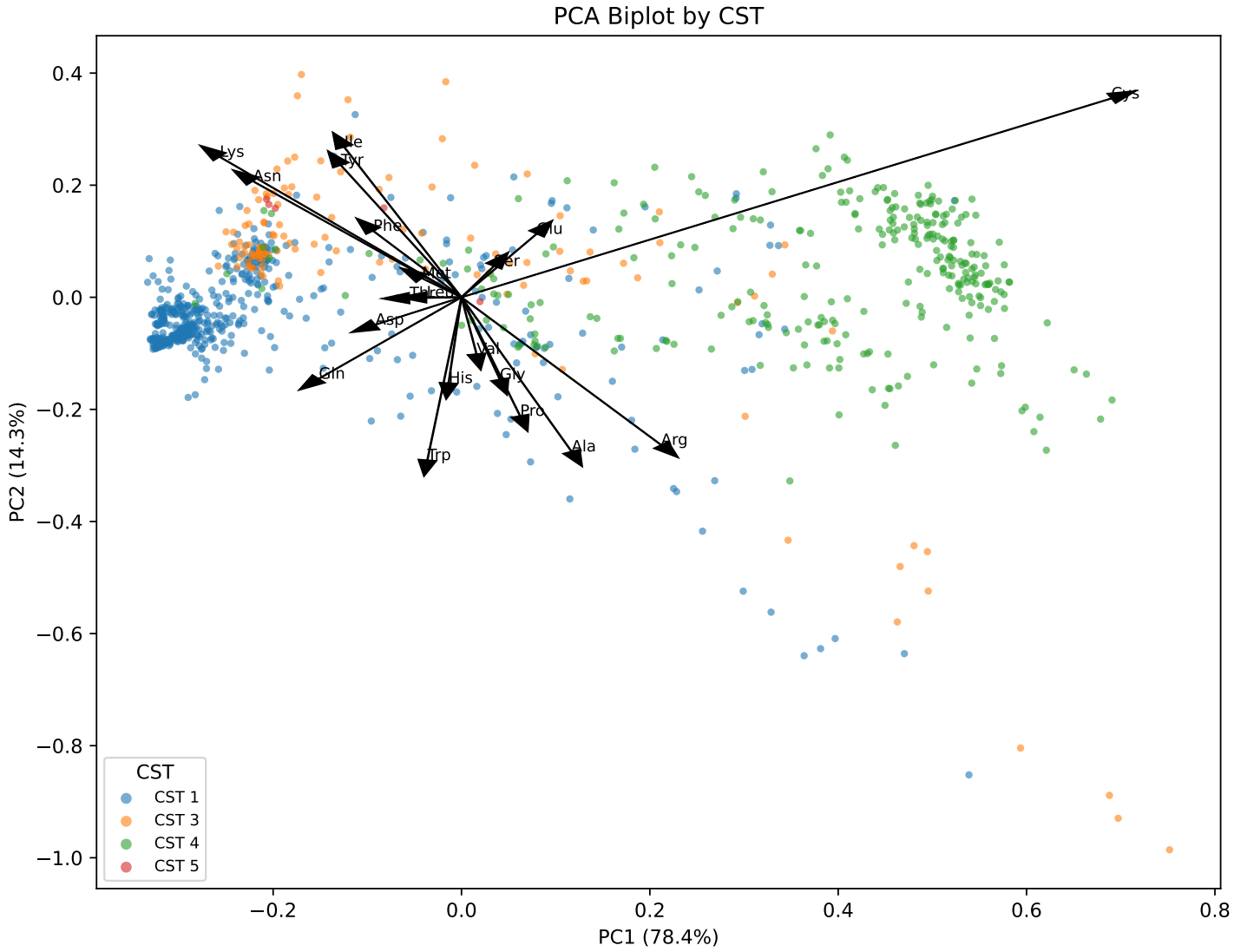
fig, ax = plt.subplots(figsize=(8, 6))
ax.scatter(load[:,0], load[:,1], alpha=0.7, s=30, linewidths=0)
for i, aa in enumerate(aa_columns):
    ax.annotate(aa, (load[i,0], load[i,1]), fontsize=9,
                xytext=(3, 3), textcoords="offset points")
ax.set_title("PCA Loadings")
ax.set_xlabel("PC1 Loading")
ax.set_ylabel("PC2 Loading")
plt.tight_layout()
plt.show()
```




```
# ==========
# PCA Biplot
# ==========

span = max(abs(X_pca[:,0]).max(), abs(X_pca[:,1]).max())
scale = 0.7*span/abs(load).max()
hw = 0.02*span

fig, ax = plt.subplots(figsize=(9, 7))
for c in cst_levels:
    m = samples["CST"] == c
    ax.scatter(X_pca[m,0], X_pca[m,1], color=cst_colors[c],
               alpha=0.6, s=14, linewidths=0, label=f"CST {c}")
for i, aa in enumerate(aa_columns):
    ax.arrow(0, 0, load[i,0]*scale, load[i,1]*scale,
             linewidth=0.7, color="black", head_width=hw)
    ax.annotate(aa, (load[i,0]*scale, load[i,1]*scale), fontsize=8)
ax.set_title("PCA Biplot by CST")
ax.set_xlabel(f"PC1 ({evr[0]:.1%})")
ax.set_ylabel(f"PC2 ({evr[1]:.1%})")
ax.legend(title="CST", markerscale=1.5, fontsize=8)
plt.tight_layout()
plt.show()
```




```
print(samples.groupby("CST")[["GC_content","Cys_pct","n_cds"]].mean())
```

```
     GC_content   Cys_pct         n_cds
CST
1     37.130178  0.665851   3675.672414
3     36.922991  0.772768   4743.114754
4     40.477388  1.125654  19429.996587
5     35.328529  0.729276   3606.000000
```

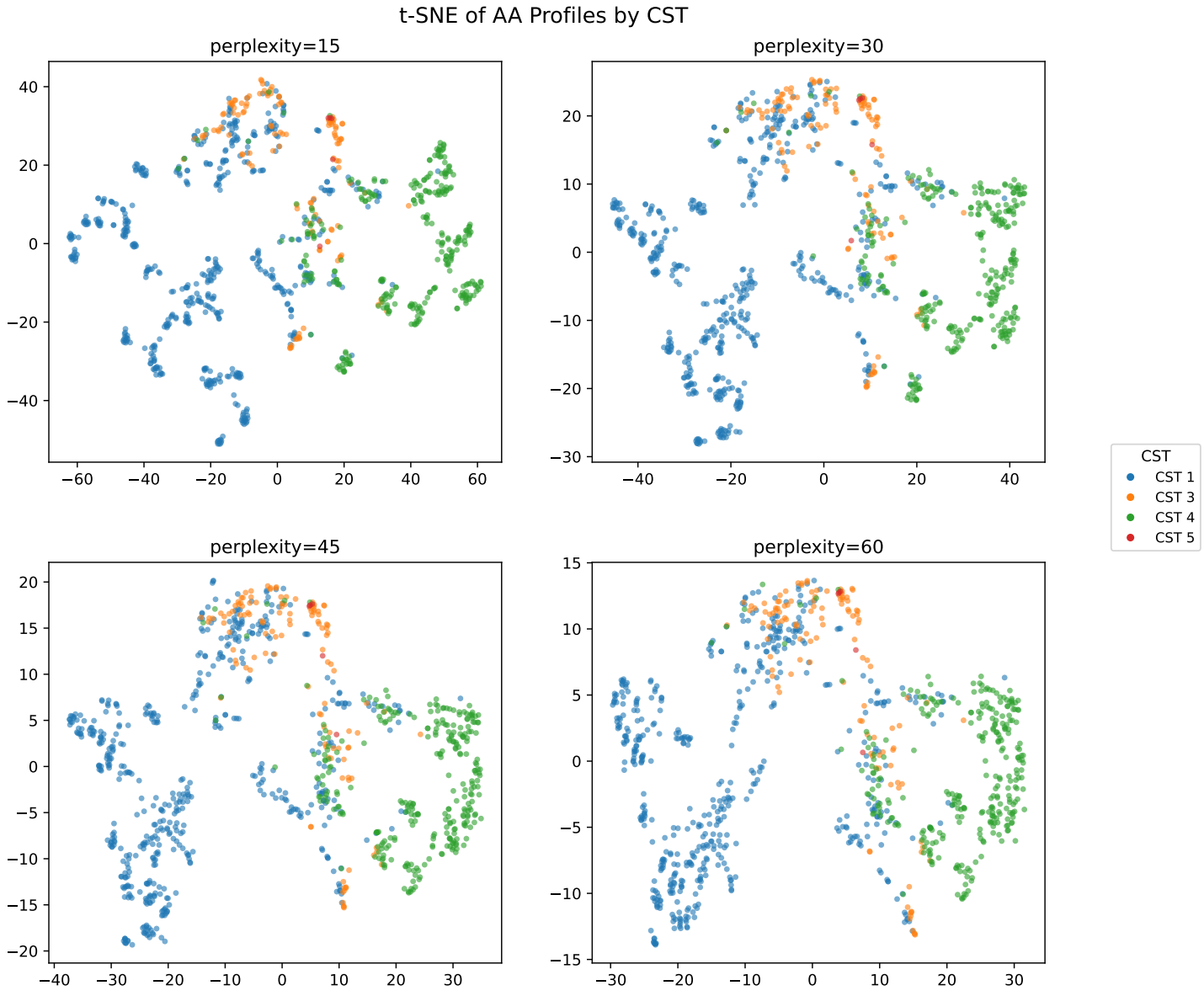
```
# ========================
# t-SNE color coded by CST
# ========================

fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes = axes.flatten()

for ax, perp in zip(axes, [15, 30, 45, 60]):
    tsne = TSNE(n_components=2, random_state=42, perplexity=perp)
    X_tsne = tsne.fit_transform(X_clr)
    for c in cst_levels:
        m = (samples["CST"] == c).values
        ax.scatter(X_tsne[m, 0], X_tsne[m, 1], color=cst_colors[c],
```

```
                    alpha=0.6, s=14, linewidths=0, label=f"CST {c}")
    ax.set_title(f"perplexity={perp}")

handles = [plt.Line2D([0],[0], marker="o", color="w",
           markerfacecolor=cst_colors[c], markersize=6, label=f"CST {c}")
           for c in cst_levels]
fig.legend(handles=handles, title="CST", loc="center right", fontsize=9)
fig.suptitle("t-SNE of AA Profiles by CST", fontsize=14)
fig.subplots_adjust(right=0.88, top=0.93, hspace=0.25, wspace=0.2)
plt.show()
```




```
# ==============
# CST STRUCTURE
# ==============

# Samples per Subject
print(samples.groupby("Subject").size().describe())

# Unique Subjects in each CST
```

```
print(samples.groupby("CST")["Subject"].nunique())

# Number of Subjects with Unique CSTs
print(samples.groupby("Subject")["CST"].nunique().value_counts())

# CST Counts
print(samples["CST"].value_counts().sort_index())
```

```
count    47.000000
mean     20.042553
std       6.730713
min       4.000000
25%      17.000000
50%      22.000000
75%      24.500000
max      27.000000
dtype: float64
CST
1    32
3    21
4    35
5     2
Name: Subject, dtype: int64
CST
2    21
1    15
3    11
Name: count, dtype: int64
CST
1    522
3    122
4    293
5      5
Name: count, dtype: int64
```

```
# =========
# PERMANOVA
# =========

D = squareform(pdist(X_clr, metric="euclidean"))
dm = DistanceMatrix(D, ids=samples["sample_id"].astype(str).values)

res = permanova(dm, samples["CST"].astype(str).values, permutations=999)
print(res)


# ========
# PERMDISP
# ========

res_disp = permdisp(dm, samples["CST"].astype(str).values, permutations=999)
print(res_disp)
```

```
method name                PERMANOVA
test statistic name         pseudo-F
sample size                      942
number of groups                   4
```

```
test statistic            375.632105
p-value                        0.001
number of permutations           999
Name: PERMANOVA results, dtype: object
method name                 PERMDISP
test statistic name          F-value
sample size                      942
number of groups                   4
test statistic             15.991755
p-value                        0.001
number of permutations           999
Name: PERMDISP results, dtype: object
```